Open Access | https://doi.org/10.25080/tcjy7995



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Adapt v0.1: Prototyping a Real-Time, Reproducible Data Analysis Framework for Adaptive Radar Scanning

Bhupendra A. Raut[1], **Siddhant Gupta**[1], **Zach Sherman**[1], **Bobby Jackson**[1], **Joe O'Brien**[1], **Eddie Schuman**[2], **Pawel Lech**[1], **Sherman Beus**[2], **Seth Abernethy**[1], **Scott Collis**[1], **Nicki Hickmon**[1], and **Adam Theisen**[1]

[1]Argonne National Laboratory, Lemont, IL, [2]Pacific Northwest National Laboratory, Richland, WA

Abstract

Adapt is a modular framework for real-time convective cell detection, motion projection, and storm lifecycle analysis, developed to support adaptive radar-scanning operations at the U.S. Department of Energy Atmospheric Radiation Measurement (DOE-ARM) User Facility. Scientific workflows are composed of independently registered modules that communicate through a shared execution context and are assembled at runtime into directed acyclic graphs (DAGs) through automatic dependency resolution. The project aims to enable plug-and-play modules and automated pipeline construction from declared dependencies. A reference NEXRAD pipeline demonstrates the architecture through data acquisition, detection, analysis, projection and tracking. A read-only data-access layer provides unified SQL and NetCDF access to pipeline outputs for real-time and archival analysis. Adapt is being developed toward a reusable ecosystem of scientific and machine-learning modules that can be composed into operational and research pipelines. Planned applications include cloud-based nowcasting dashboards and JupyterHub workflows on DOE-ARM computing infrastructure. We welcome contributions in architecture, validation, software design, and governance to strengthen scalability and maintainability. As AI-assisted development becomes increasingly prevalent, a central objective is to build an architecture that is explicit, verifiable, and resilient while supporting both human developers and coding agents.

Keywords convective storms, weather satellites, remote sensing, cell detection and tracking, nowcasting, data pipelines, scientific workflows

Published Jul 13, 2026

Correspondence to
Bhupendra A. Raut
braut@anl.gov

Open Access

Copyright © 2026 Raut *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Adaptive radar scanning enables real-time response to evolving convective storms by focusing radar on dynamically selected sectors [1]. Traditional scanning strategies must balance spatial coverage, temporal frequency, and vertical resolution under fundamental physics constraints, frequently undersampling rapid microphysical evolution during convective lifecycles. Real-time feedback control offers a pathway to dramatically improve observations of transient phenomena but demands low-latency analysis, reliable decision support, and infrastructure that remains stable under continuous operation [2], [3].

The TITAN system demonstrated operational radar data analysis in real time as early as the 1990s [4]. Modern open-source frameworks such as PyART, tobac, TINT, and CoCo-MET provide powerful scientific algorithms primarily designed for offline batch workflows [5], [6], [7], [8]. Composing them into a reliable, continuously running, operationally maintainable pipeline introduces requirements that go beyond scientific algorithms: the need is

continuous data ingestion, event-driven execution, configurable processing chains without code changes, separation of consumers from producers, and long-term reproducibility.

Adapt aims to address these requirements through a layered architecture in which scientific modules are registered, wired by dependency, and executed by a graph-based orchestrator. The current implementation processes NEXRAD Level-II data in real time or archival mode, writing validated outputs to a structured repository that external consumers access through a read-only API.

This paper presents the design of Adapt for scientific developers seeking to extend the framework with new modules, pipelines, or data consumers. System Design introduces the layered architecture, followed by the module system (Plug-and-Play Module System), execution graph (Directed Acyclic Graph Execution), configuration system (Three-Tier Configuration), and data repository and consumer API (Data Repository and Consumer API). NEXRAD Pipeline presents the reference NEXRAD pipeline, Scientific Python Stack summarizes the scientific Python software stack, and Roadmap and Contribution Opportunities outlines future development and contribution opportunities.

2. System Design

Adapt is structured in layers, each with a single, well-defined responsibility. Layers communicate through explicit interfaces as shown in Figure 1.

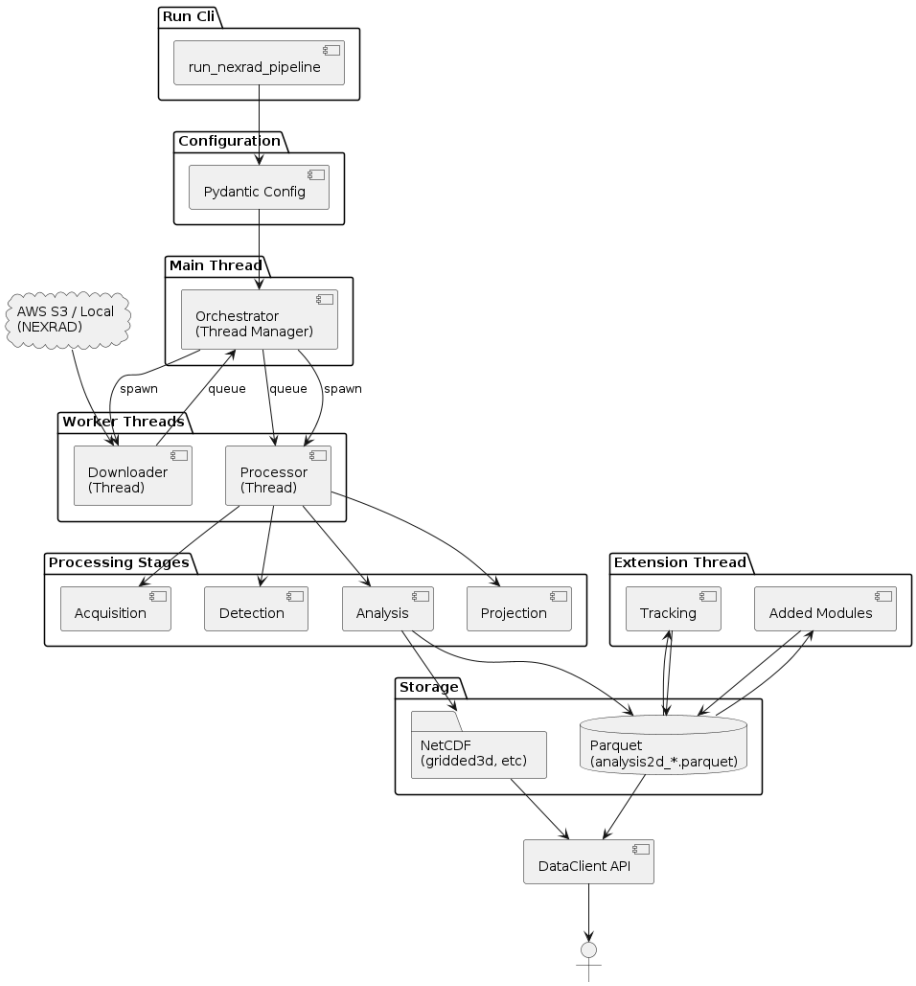


Figure 1. *Adapt execution workflow. A two-thread runtime (downloader and processor) drives a DAG-based execution graph. Scientific modules are registered plugins that communicate through a shared context dictionary. All outputs are written to a structured repository accessed through a read-only consumer API.*

2.1. Layer Responsibilities

Module layer contains all scientific algorithms. Modules are stateless functions wrapped in a standard interface and they do not have I/O, threading, or repository access. Each module declares the named data products it reads from and writes to a shared context dictionary. The graph executor wires and sequences them automatically.

Execution layer hosts `GraphBuilder`, which constructs a DAG from module input/output declarations, and `GraphExecutor`, which traverses the graph in topological order, enforcing contracts at every module boundary. The execution layer is the only layer that directly instantiates modules. It does so through the module registry, not by importing module classes.

Runtime layer coordinates threads and data flow. It starts and stops the downloader and processor threads, owns the inter-thread queue, manages the two-frame rolling buffer required by the projection module, and delegates each file to the pipeline. No scientific logic resides in the runtime layer.

Persistence layer writes and indexes all artifacts to the local data repository. Gridded fields are stored as NetCDF and tabular analysis outputs as Parquet while tracking data and metadata catalogs as SQLite. All file paths are encapsulated within this layer and no other layer constructs paths or directory structure directly.

Configuration layer resolves a three-tier hierarchy into a single immutable `InternalConfig` object at startup. No component reads configuration files at runtime.

Consumer API layer provides a read-only API (`DataClient`) that wraps catalog queries, DuckDB-powered Parquet SQL, and a polling-based streaming interface. Dashboards, notebooks, and autonomous pipelines can use only this layer and they have no access to the pipeline's internal state.

2.2. Dependency Rules

The import rules are enforced by `import-linter` and checked in CI, so a layering violation fails the build even when every unit test passes.

3. Plug-and-Play Module System

The module system will be the primary extension point for scientific developers. New modules can be integrated and tested without modifying any existing source files.

3.1. Module Interface

Every processing module subclasses `BaseModule` and declares four class-level attributes:

```python
class MyModule(BaseModule):
    name = "my_module"
    inputs = ["grid_ds_2d", "config"]
    outputs = ["my_product"]

    def run(self, context: dict) -> dict:
        data = context["grid_ds_2d"]
        result = compute(data)
        return {"my_product": result}
```

Program 1. *Minimal module definition.*

Optionally, a module may declare `input_contracts` and `output_contracts` as dictionaries mapping context keys to validator functions and also their version. The graph executor enforces these contracts automatically. The module author does not call input/output validation logic.

3.2. Registration

Modules register with a global `ModuleRegistry` when their package is imported. Registration records the module class by its `name` attribute. The pipeline configuration specifies which module packages to import, the registry instantiates them, and the graph builder wires them automatically. Adding a new module to a pipeline requires:

1. Implement `BaseModule` in `modules/<domain>/module.py`
2. Currently requires adding the package path to the `pipeline.modules` list in the pipeline's configuration file
3. Optionally declare contracts for boundary validation. These implementations will be made mandatory in the stable release.

No other source files should require modification for adding a module. Replacing an existing module (for example, substituting a deep-learning segmenter for the threshold-based detector) requires only that the replacement produce the same output context key (`segmented_ds`) satisfying the same output contract.

3.3. Scaling to Many Modules

Because modules communicate only through context keys and are wired by the graph builder from their declarations, a large number of modules can coexist in the registry without interfering. Pipeline configurations select which registered modules to activate, and inactive modules have zero runtime cost.

Table 1 lists the modules in the current NEXRAD reference pipeline. New module categories currently under consideration include: wavelet-based cell detection, lightning data ingestion, GOES satellite ingest, nowcasting tendency modules, and alert-generation modules for automated scan-control decisions.

4. Directed Acyclic Graph Execution

4.1. Graph Construction

`GraphBuilder` constructs the execution DAG during pipeline initialisation. It creates a node for each registered module, builds an output map from product names to producing nodes, and wires dependencies by matching each module's declared inputs to entries in the output map. Two checks are enforced at construction time: (1) all declared inputs must have a corresponding producer, and (2) no two modules may declare the same output key. Violations cause startup failure before any processing begins.

4.2. Execution and Context Dictionary

`GraphExecutor` traverses the graph in topological order within a single thread. At each step it selects nodes whose upstream dependencies have completed, invokes the module, and merges returned outputs into the shared execution context dictionary. The context dictionary holds all in-memory data products for one file's execution and is discarded after the file completes.

Input contracts are enforced before each module invocation and output contracts after the module finishes processing. A `ContractViolation` halts processing for the current file and records a diagnostic failure state in the catalog; processing continues with subsequent files.

4.3. Multiple Pipelines

A pipeline is a named configuration that specifies which modules to activate and in what parameter regime. Because module wiring is derived automatically from declarations, a new pipeline definition currently requires only a configuration file listing the desired modules. The same module can participate in multiple pipelines. Currently, the NEXRAD pipeline is the only implementation and we aim to provide a framework for additional pipelines (e.g., for satellite data, multi-sensor fusion, or model data).

This design can scale naturally to tens of pipelines. Users select a pipeline by name via CLI flag or configuration file.

5. Three-Tier Configuration

Pipeline behaviour is controlled through a layered configuration hierarchy. The lowest tier is `ParamConfig`, encoding defaults for every tunable parameter, maintained in a `defaults.yaml`. Users must not edit this file. The middle tier is `UserConfig`, a config file containing only what they wish to override in `ParamConfig`. The highest tier is `CLIConfig`, populated from command-line flags.

At startup, `resolve_config()` merges the three tiers in precedence order (CLI overrides user, user overrides param) and validates the result as a frozen `InternalConfig` object. Unknown fields in user files raise a `ValidationError` immediately. The resolved configuration is stored alongside pipeline outputs, creating a permanent record sufficient to reproduce any run.

This design separates scientific expertise (defaults), deployment preferences (user config), and operational flags (CLI) into independent, composable concerns. A shared configuration file captures pipeline settings and algorithm parameters for reproducibility.

A user config lists the modules to run and overrides only the defaults it cares about; every unspecified parameter falls back to `defaults.yaml`.

```yaml
base_dir: /path/to/repository
mode: historical
source: aws_nexrad
modules:            # dependency order; subset at runtime with --only / --not
  - ingest
  - detection
  - projection
  - analysis
  - tracking
  - cell_volume_stats
regridder:
  grid_shape: [41, 301, 301]
  grid_limits: [[0, 20000], [-150000, 150000], [-150000, 150000]]
  weighting_function: cressman
  save_netcdf: true
```

Program 2. *Excerpt of a user `config.yaml`. Only overrides are listed; all other parameters inherit from `defaults.yaml`.*

The pipeline is generated and run from the command line. `adapt config` writes a template, and `run-nexrad` processes a small historical window for one radar:

```
adapt config                                # write a config.yaml template
adapt run-nexrad config.yaml --radar KHTX \
    --mode historical \
    --start-time 2026-07-24T12:00 --end-time 2026-07-24T13:00
```

Program 3. *Generating a config template and running the NEXRAD pipeline on a historical case.*

CLI flags override the config file, so a single shared config can be reused across runs while `--radar`, the time window, `--only`/`--not` (to activate a subset of modules), and `--max-runtime` (realtime mode) vary per invocation.

6. Contract-Based Validation

Scientific pipelines are susceptible to silent failures when intermediate data products change structure. Adapt addresses this through a contract system that validates data products at module boundaries. Contracts are pure functions in module `contracts.py` files that verify invariants such as: variable presence, array dimensionality, integer label integrity, coordinate consistency, and DataFrame schema completeness.

Each module may declare optional `input_contracts` and `output_contracts` dictionaries that map a context key to a `(validator, version)` pair. The graph executor calls these automatically and the module author does not invoke validation logic. For example, the detection module guarantees that its `segmented_ds` output satisfies the segmentation contract:

```python
from adapt.modules.detection.contracts import check_segmentation

class Detection(BaseModule):
    name = "detection"
    inputs = ["grid_ds_2d", "config"]
    outputs = ["segmented_ds"]
    output_contracts = {"segmented_ds": (check_segmentation, "v1")}
```

Program 4. *A module declares contracts by name; the executor enforces them at the boundary.*

Here `check_segmentation` is a pure function in `detection/contracts.py` that verifies the segmentation invariants (label variable present, integer dtype, non-negative labels, 2-D shape). Because the guarantee is attached to the `segmented_ds` key rather than to the detector implementation, a deep-learning segmenter can replace the threshold-based detector as long as it satisfies the same output contract. Failures raise `ContractViolation` with a message

identifying which contract failed, what was wrong, and which module produced the data. Contract coverage can be introduced incrementally as module interfaces stabilise.

Contracts currently do not validate scientific correctness or statistical properties. As these are monitoring concerns, not interface contracts, checking them can be added as separate layers around the science modules, considering legitimate edge cases such as unusual storm morphologies or degraded radar modes.

7. Data Repository and Consumer API

7.1. Repository Structure

The repository is a structured directory tree combining SQLite catalog tables with per-scan NetCDF and Parquet files, organised under a fixed layout. All artifacts are written through `DataRepository`, the single component that owns this layout, rather than by modules opening files or constructing paths directly. This indirection keeps producers agnostic to storage details and lets the consumer API resolve and read only the fields a query needs. Migrating the underlying store (for example, to a Zarr or Icechunk backend) therefore requires no changes to the science modules. Gridded outputs (radar volumes and segmentation) are stored as NetCDF [9] files per scan with zlib compression. Per-cell statistics are stored as Parquet files for efficient columnar queries. Cell tracking data is stored in per-radar SQLite tables to support row-oriented graph traversal queries (single storm history, recent events, lineage enumeration). We are currently exploring cloud-native data-store strategies such as Zarr, a chunked format for large N-dimensional arrays [10], and Icechunk, a transactional storage engine layered on Zarr, for flexible concurrent read-write and long-term cloud storage capabilities. The repository layout and its access APIs are still stabilising ahead of the v1.0 release, and consumers should expect breaking changes until then.

7.2. Reproducibility and Run Management

Each processing run receives a unique identifier of the form `YYYYMONDD-HHMM-RADAR`. Cell identifiers (`cell_uid`) are deterministic hashes derived from observable properties at initiation time. Hence the same input always produces the same identifiers regardless of when or how many times processing is performed. Reprocessing historical datasets is identical to real-time operation.

7.3. Consumer API

`DataClient` is the sole interface through which downstream systems access pipeline outputs. No consumer has access to `DataRepository`, `RadarCatalog`, or any pipeline internal. The interface provides:

```
client = DataClient("/path/to/output")

# Batch queries via DuckDB over Parquet
df = client.query(
    "SELECT * FROM analysis "
    "WHERE cell_area_sqkm > 50"
)

# Streaming for live dashboards
for df in client.stream(
    "SELECT * FROM cells_by_scan "
    "WHERE time > :last_seen",
    poll_interval=5
):
    update_dashboard(df)
```

Program 5. *Consumer API usage.*

DuckDB executes SQL directly against Parquet files with predicate and projection push-down, making queries over multi-month archives feasible without loading data into memory. The streaming interface yields new rows as scans complete, enabling live dashboards to update incrementally without filesystem access or pipeline coupling. This producer/consumer separation is fundamental to the architecture.

8. NEXRAD Pipeline

The NEXRAD pipeline demonstrates the full architecture end-to-end. It processes NEXRAD Level-II data containing reflectivity, radial velocity, and spectrum width for each radar volume scan [11], through six registered modules (shown in Table 1) and exposes results through `DataClient`. The Level-II archive is publicly hosted on AWS S3 as an Open Data registry dataset (`s3://noaa-nexrad-level2`)[1].

8.1. Two-Thread Runtime

The pipeline operates as two cooperating threads. `AwsNexradDownloader` continuously polls S3 for new Level-II files (realtime mode) or enumerates a specified time range and exits (historical mode). Files are placed in a bounded queue providing back-pressure when the processor is slower than the downloader. `RadarProcessor` dequeues files, checks for duplicates, maintains a two-frame rolling buffer for the projection module, executes the DAG via `NexradPipeline`, and writes results through `RepositoryWriter`. No shared mutable state exists between threads beyond the queue.

Table 1. *Modules in the NEXRAD reference pipeline, in dependency order. Each module reads and writes named products in the shared context; the graph builder derives the execution order from these declarations.*

Module	Inputs arrow.r Outputs	Function and Data Flow
`ingest`	Level-II file arrow.r gridded volume	Downloads Level-II files from AWS S3 to a background queue, decodes them with PyART, and regrids to a Cartesian volume.
`detection`	gridded volume arrow.r labeled cells	Segments 2D grids into labeled storm-cell datasets (`segmented_ds`) via thresholding and connected-component labeling.
`projection`	current + previous labeled cells arrow.r projected boundaries	Uses Farnebäck optical flow on current and past segments to project cell boundaries forward.
`analysis`	labeled cells + projected boundaries arrow.r per-cell statistics	Combines grids and projected boundaries into 2D per-cell statistics.
`tracking`	per-cell statistics arrow.r tracking graph	Links cells across scans, generating tracking graphs and lifecycle (split/merge) events.
`cell_volume_stats`	gridded volume + labeled cells arrow.r 3D cell statistics	Computes 3D volume statistics (e.g. cloud-top height) per cell; requires `regridder.save_netcdf`.

8.2. Runtime Performance

Table 2 reports per-module timings from a sample realtime run over the KHTX radar (`adapt run-nexrad config.yaml --radar KHTX`), processing six volume scans (about 10 MB per

[1] https://registry.opendata.aws/noaa-nexrad/

Level-II file) and detecting 343 cell objects on an Apple M3 Mac with 36 GB RAM. End-to-end processing averaged 19.0 s per scan (max 20.04 s), dominated by ingestion (download, decode, and regridding). NEXRAD volume scans complete roughly every four to six minutes depending on the operational scan strategy, so a per-scan cost of about 19 s leaves ample headroom for real-time operation.

Table 2. *Per-module timings for a six-scan KHTX sample run (343 objects detected).*

Module	Calls	Total (s)	Avg (s)
`ingest`	6	79.0	13.2
`detection`	6	14.2	2.4
`projection`	5	18.8	3.8
`analysis`	5	0.71	0.14
`tracking`	5	1.1	0.22
`cell_volume_stats`	5	21.0	4.2

9. Scientific Python Stack

Xarray. All gridded data products are `xr.Dataset` objects. Spatial dimensions (z, y, x), coordinate information, physical units, and timestamps travel with the data through every module. No function in the pipeline accepts grid spacing, CRS, or coordinate names as arguments; this information must be embedded in the dataset [12].

PyART. NEXRAD Level-II decoding and Cartesian gridding are currently performed through PyART [5]. PyART is the core radar library because it supplies a broad range of radar algorithms except tracking (e.g. I/O, gridding, and classification). Adapt's own detection uses a tobac-like segmentation, and its tracking is an improved variant of TINT [7] adapted for real-time operation. At present, PyART usage is largely isolated to the ingestion and gridding stages. However, PyART is expected to become a foundational dependency within Adapt, with future modules exposing PyART-based radar analysis algorithms directly through configuration. This approach will allow users to construct processing pipelines from a broad range of PyART capabilities without modifying framework code.

Pydantic. Configuration validation and schema enforcement. Expert defaults, user overrides, and CLI flags are resolved and frozen into a single `InternalConfig` at startup. Unknown fields in user configurations raise at startup; type and range violations produce descriptive errors before any processing begins [13].

OpenCV and SciPy. Dense optical flow via the Farnebäck algorithm (OpenCV) for motion estimation. Connected component labeling and morphological operations via `scipy.ndimage`. Linear sum assignment for optimal cell-to-cell matching via `scipy.optimize` [14], [15].

NetworkX. The tracking module maintains a `networkx.DiGraph` of all observed cells and lineage edges across the processing session. This graph computes cell age and dominant lineage edges in split/merge complexes [16].

DuckDB and PyArrow. Per-cell statistics are persisted as Parquet, a columnar on-disk file format for analytical data [17], written via PyArrow, the Python bindings to Apache Arrow's in-memory columnar layout [18]. Downstream queries are executed by an embedded DuckDB instance within `DataClient`. DuckDB is an in-process analytical SQL engine that runs directly against Parquet files [19] and applies predicate and projection pushdown without loading full datasets into memory.

10. Roadmap and Contribution Opportunities

Adapt is an actively developed prototype. Table 3 summarizes what is implemented today, what is experimental, and what is planned, to distinguish the working system from its intended trajectory.

Table 3. *Implementation status of Adapt components.*

Status	Components
Implemented	NEXRAD pipeline (six modules), registry-based module system, dependency-driven DAG executor, three-tier configuration, read-only `DataClient` access layer, opt-in boundary contracts, `postprocess` command for repository enrichment, and `cell_volume_stats` 3D statistics (cloud-top height).
Experimental	Migration of the data repository to a Zarr/Icechunk cloud-native store.
In progress	Tracking of connected groups of cells (storm complexes).
Planned	Offline post-processing engine for after-run analysis; parallel satellite and radar pipelines with product merging; ML-based nowcasting trained on archived Adapt output; 3D modules for segmentation and storm analysis; mandatory contracts at the stable release.

The first stable release (v1.0) will establish the core architectural foundations of Adapt including a registry-based module system, dependency-driven DAG execution, configuration-defined pipelines, and a stable data-access layer. Current development focuses on simplicity, composability, and maintainability so that future capabilities can be added without changes to the framework core.

At this stage, our highest priority is architectural feedback. We welcome design reviews, validation strategies, testing frameworks, and software engineering practices that improve scalability, correctness, extensibility, and long-term sustainability.

After v1.0, community contributions will expand toward new modules, algorithms, pipeline definitions, data products, and user-facing applications.

10.1. Architecture and Agentic Development

Adapt is being developed at a time when scientific software is increasingly created and maintained using AI-assisted and agentic workflows. To preserve architectural integrity, the project already employs automated checks including `pytest`, `import-linter`, static type checking with `mypy`, and code-quality tools. These tools provide a first line of defense against architectural drift and unintended design violations introduced during rapid development.

We seek contributions on additional mechanisms for architecture validation and governance, including interface contracts, architecture-aware testing, machine-readable design specifications, dependency analysis, automated design-rule enforcement, and other approaches that make architectural constraints explicit and verifiable.

10.2. Pipeline Composition and Orchestration

Adapt constructs execution graphs from declared module dependencies, but pipeline definitions remain largely static. Future work will explore more flexible pipeline composition from available modules, data products, and instrument platforms. Contributions are encouraged in dependency resolution, workflow planning, execution scheduling, and multi-instrument workflows.

10.3. Cloud and Interactive Applications

Future deployments include cloud-hosted nowcasting services, web-based dashboards, and JupyterHub environments for scientific analysis. Contributions in deployment architecture, distributed execution, cloud storage, and data-serving infrastructure are welcome.

11. Summary

Adapt establishes the architectural foundations for a modular framework supporting adaptive radar-scanning operations. Independently developed modules are composed automatically through dependency-driven DAG execution, enabling extensible and reproducible scientific workflows.

The reference NEXRAD pipeline demonstrates the architecture from data acquisition to cell tracking, while the consumer API provides a stable interface for downstream analysis and applications. Future development will focus on strengthening the architecture, expanding the module ecosystem, and supporting new scientific workflows and instrument platforms.

Code and documentation are available at https://github.com/ARM-DOE/adapt.

Acknowledgments

This research was supported by the Atmospheric Radiation Measurement (ARM) User Facility, a U.S. Department of Energy Office of Science user facility managed by the Biological and Environmental Research Program. Argonne National Laboratory's work was supported by the U.S. Department of Energy, Office of Science, Office of Biological and Environmental Research, under Contract DE-AC02-06CH11357.

The work benefited from interactions with Sean Shahkarami, Sean Freeman, Brenda Dolan, Michael Giansiracusa, and Scott Giangrande. The authors also thank Jim Mather, Jennifer Comstock, Giri Prakash, and the DOE-ARM radar community for their support and feedback. Valuable discussions with the tobac development team and the Open Radar community also contributed to the direction of this work.

References

[1] M. Oue, S. M. Saleeby, P. J. Marinescu, P. Kollias, and S. C. Van Den Heever, "Optimizing radar scan strategies for tracking isolated deep convection using observing system simulation experiments," *Atmospheric Measurement Techniques*, vol. 15, no. 16, pp. 4931–4950, 2022, doi: 10.5194/amt-15-4931-2022.

[2] K. Lamer, P. Kollias, E. P. Luke, B. P. Treserras, M. Oue, and B. Dolan, "Multisensor agile adaptive sampling (MAAS): A methodology to collect radar observations of convective cell life cycle," *Journal of Atmospheric and Oceanic Technology*, vol. 40, no. 11, pp. 1509–1522, 2023, doi: 10.1175/JTECH-D-22-0148.1.

[3] S. Gupta and A. Theisen, "A Cloud-Tracking Data Set for the CSAPR2 Adaptive Scanning during TRACER," techreport DOE/SC-ARM-TR-324, 2025. doi: 10.2172/3001951.

[4] M. Dixon and G. Wiener, "TITAN: Thunderstorm identification, tracking, analysis, and nowcasting—A radar-based methodology," *Journal of Atmospheric and Oceanic Technology*, vol. 10, no. 6, pp. 785–797, 1993, doi: 10.1175/1520-0426(1993)010<0785:TTITAA>2.0.CO;2.

[5] J. J. Helmus and S. M. Collis, "The Python ARM Radar Toolkit (Py-ART), a library for working with weather radar data in the Python programming language," *Journal of Open Research Software*, vol. 4, 2016, doi: 10.5334/jors.119.

[6] G. A. Sokolowsky *et al.*, "tobac v1.5: Introducing fast 3D tracking, splits and mergers, and other enhancements for identifying and analysing meteorological phenomena," *Geoscientific Model Development*, vol. 17, no. 13, pp. 5309–5330, 2024, doi: 10.5194/gmd-17-5309-2024.

[7] B. A. Raut, R. Jackson, M. Picel, S. M. Collis, M. Bergemann, and C. Jakob, "An adaptive tracking algorithm for convection in simulated and remote sensing data," *Journal of Applied Meteorology and Climatology*, vol. 60, no. 4, pp. 513–526, 2021, doi: 10.1175/JAMC-D-20-0119.1.

[8] T. Hahn, H. Weiner, C. Brooks, J. X. Li, S. Gupta, and D. Wang, "CoCoMET v1.0: a unified open-source toolkit for atmospheric object tracking and analysis," *Geoscientific Model Development*, vol. 18, no. 18, pp. 5971–5996, 2025, doi: 10.5194/gmd-18-5971-2025.

[9] R. Rew and G. Davis, "NetCDF: an interface for scientific data access," *IEEE Computer Graphics and Applications*, vol. 10, no. 4, pp. 76–82, 1990, doi: 10.1109/38.56302.

[10] J. Moore and S. Kunis, "Zarr: A cloud-optimized storage for interactive access of large arrays," in *Proceedings of the Conference on Research Data Infrastructure*, 2023. doi: 10.52825/cordi.v1i.285.

[11] NOAA National Weather Service (NWS) Radar Operations Center, "NOAA Next Generation Radar (NEXRAD) Level 2 Base Data." NOAA National Centers for Environmental Information, 1991. doi: 10.7289/V5W9574V.

[12] S. Hoyer and J. Hamman, "xarray: ND labeled Arrays and Datasets in Python," *Journal of Open Research Software*, vol. 5, no. 1, 2017, doi: 10.5334/jors.148.

[13] P. K. Narayanan, "Getting Started with Data Validation using Pydantic and Pandera," in *Data Engineering for Machine Learning Pipelines: From Python Libraries to ML Pipelines and Cloud Platforms*, Springer, 2024, pp. 163–196.

[14] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[15] G. Farnebäck, "Two-frame motion estimation based on polynomial expansion," in *Scandinavian Conference on Image Analysis*, 2003, pp. 363–370. doi: 10.1007/3-540-45103-X_50.

[16] A. Hagberg, P. Swart, and D. S Chult, "Exploring network structure, dynamics, and function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, Pasadena, CA, USA, 2008. doi: 10.25080/TCWV9851.

[17] D. Vohra, "Apache Parquet," in *Practical Hadoop Ecosystem: A Definitive Guide to Hadoop-Related Frameworks and Tools*, Berkeley, CA: Apress, 2016, pp. 325–335.

[18] G. Lentner, "Shared memory high throughput computing with Apache Arrow," in *Practice and Experience in Advanced Research Computing 2019: Rise of the Machines (learning)*, 2019, pp. 1–2. doi: 10.1145/3332186.3333156.

[19] M. Raasveldt and H. Mühleisen, "DuckDB: an embeddable analytical database," in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 1981–1984. doi: 10.1145/3299869.3320212.